

Copy the following code inside main(), place a breakpoint in front of the counter declaration and start a debug session:

```
int main()
{
    int counter = 0;
    counter++;
    counter++;
    counter++;
    counter++;
    counter++;
    counter++;
    counter++;
    counter++;
    counter++;
    counter++;
}
```

Put a breakpoint at the first counter++; line and execute your project. Switch to the Registers view and set the counter value manually to 0x7FFFFFFF.

Question 2.1: How do you determine which register holds the value of the *counter* variable?

Question 2.2: Which number in decimal corresponds to 0x7FFFFFFF ?

Question 2.3: Now step over the code once (F5):

- What is the new value stored in the register?
- Why?
- What happens if you step more?
- What is the binary representation of 0x00? What happens if you subtract one?

Question 2.4: Change the declaration of the counter to register unsigned int and execute the same steps above. What do you notice?

Now create a new counter, counter2 and use a while loop to increment it:

```
register int counter2 = 0;
while (counter2 < 10) {
    counter2++;
}
```

We will have a look at the Disassembly view to examine what is going on:

```
29 register int counter2
0800029e: movs    r5, #0
30      while (counter2 < 10) {
080002a0: b.n     0x80002a4 <main+44>
31      counter2++;
080002a2: adds    r5, #1
30      while (counter2 < 10) {
```

```
080002a4: cmp    r5, #9
080002a6: ble.n  0x80002a2
```

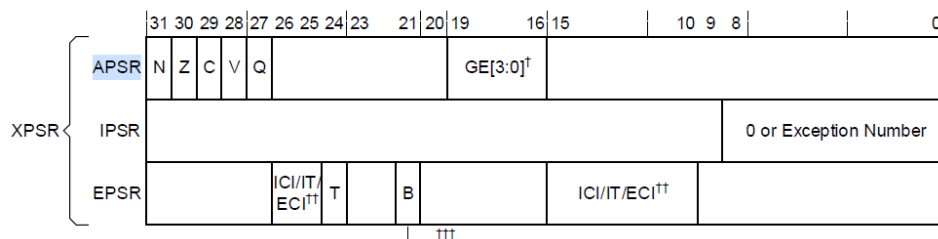
The while loop begins with an *unconditional branch* instruction - **b** (.n means that this is a 16-bit instruction) to address 0x80002a4:

B <label>

In address 0x80002a4 the CMP instruction is executed to update the APSR flags on the subtraction of Rn – imm:

CMP Rn, #<imm>

CMP is interesting in that it modifies the Application Program Status Register (APSR) register - specifically the N bit (The APSR, IPSR, and EPSR combine to form one register, the XPSR):

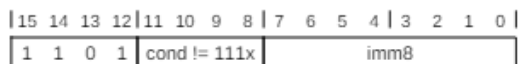


- N is set to 1 when the result of the operation was negative
- Z is set to 1 when the result of the operation was zero
- C is set to 1 when the operation resulted in a carry
- V is set to 1 when the operation caused overflow

The Branch if Less than or Equal instruction (BLE.N) will cause the program to jump back to the 0x80002a2 address if the Zero flag (Z) is set or the Negative flag (N) is not equal to the Overflow flag (V) after the last arithmetic operation. It has the following format (p568):

T1

Armv8-M



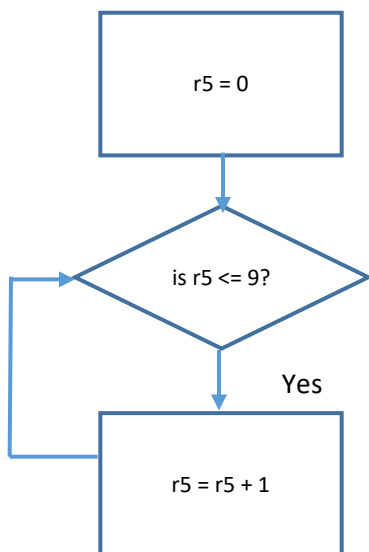
cond takes its values from this table (p463):

cond	Mnemonic extension	Meaning, integer arithmetic	Meaning, Floating-point arithmetic	APSR condition flags
0000	EQ	Equal	Equal	Z == 1
0001	NE	Not equal	Not equal, or unordered	Z == 0
0010	CS	Carry set	Greater than, equal or unordered	C == 1
0011	CC	Carry clear	Less than	C == 0
0100	MI	Minus, negative	Less than	N == 1
0101	PL	Plus, positive or zero	Greater than, equal or unordered	N == 0
0110	VS	Overflow	Unordered	V == 1
0111	VC	No overflow	Not unordered	V == 0
1000	HI	Unsigned higher	Greater than or unordered	C == 1 and Z == 0
1001	LS	Unsigned lower or same	Less than or equal	C == 0 or Z == 1
1010	GE	Signed greater than or equal	Greater than or equal	N == V
1011	LT	Signed less than	Less than or unordered	N != V
1100	GT	Signed greater than	Greater than	Z == 0 and N == V
1101	LE	Signed less than or equal	Less than, equal or unordered	Z == 1 or N != V
1110	None (AL)	Always (unconditional)	Always (unconditional)	Any

The immediate encodes the signed offset of the target address from the current PC (i.e. this instruction + 8) - note that this offset is counted in words, not bytes; since instructions are always word-aligned, the bottom two bits of any offset will always be 00 and are thus made implicit in the encoding.

FC = -4 in **two's complement** so the PC will be offset by 4! (from 64a to 646, back to the adds instruction!)

Note: The assembly loop does not necessarily correspond *exactly* to the c code - the compiler can produce different (and sometimes better) code for us. Can you visualize the assembly loop above using a diagram?



Note: Loops introduce an overhead due to the additional instructions:

```

08000646:  adds    r4, #1
163      while (counter < 10) {
08000648:  cmp     r4, #9
0800064a:  ble.n   0x8000646 <main+70>

```

It gets worse.. Jumps add additional executional delays due to **pipeline stalls**

Now move the definition for *int counter2* outside of main (without the *register identifier*) - counter2 is no longer a local variable. Go to the Expression tab and add 'counter2' to watch list. Then go to the Memory Browser tab and add the address to the counter variable '&counter2'. You can see that the variable is stored in location 0x200xxx in SRAM. You can also input this address in the Memory tab to verify it!

From the Arm Cortex-M33 Devices Generic User Guide:

The Cortex-M33 processor has a fixed default memory map that provides up to 4GB of addressable memory:

Vendor-specific memory	511MB	0xFFFFFFFF
Private peripheralbus	1MB	0xE0100000 0xE00FFFFF
External device	1.0GB	0xE0000000 0xDFFFFFFF
External RAM	1.0GB	0xA0000000 0x9FFFFFFF
Peripheral	0.5GB	0x60000000 0x5FFFFFFF
SRAM	0.5GB	0x40000000 0x3FFFFFFF
Code	0.5GB	0x20000000 0x1FFFFFFF
		0x00000000

Now let's step through the while loop watching the value of counter. Note that its value is updated after the STR instruction is executed. The assembly code of the while structure is now:

```

counter++
ldr    r3, [pc, #48] @ (0x8000708 <main+264>)
ldr    r3, [r3, #0]

```

```
adds r3, #1
ldr r2, [pc, #40] @ (0x8000708 <main+264>)
str r3, [r2, #0]
```

```
while (counter2 < 10) {
ldr r3, [pc, #36] @ (0x8000708 <main+264>)
ldr r3, [r3, #0]
cmp r3, #9
ble.n 0x80006d6 <main+214>
```

LDR (Load word) loads a word from memory to a register. [pc, #48] is called PC-relative addressing.

- 1) **ldr r3, [pc, #48]**: PC = 0x80006d6, therefore PC + 48 = 0x0800 0706. The content of 0x08000708 = E408 and 0x200008E4 is the location of the actual counter variable
- 2) **ldr r3, [r3, #0]**: Load to R3 the value stored in the *memory location* that is currently stored in R3 (+ offset 0) .
- 3) **adds r3, #1**: increases the value stored in the R3 register by 1
- 4) **ldr r2, [pc, #40]**: loads the address of the counter *var* to register R2
- 5) **str r3, [r2, #0]**: stores the value of R3 to the address stored in R2 (the address of counter)

This is the Load-Store memory model of the RISC architecture where memory can only be accessed by load and store instructions and not manipulated directly. All operations must be performed in the registers.

Exercise 1:

Given an integer num = 0xD7, write a C program to:

- Set the 2nd bit (counting from 0, LSB)
- Clear the 5th bit
- Toggle the 6th bit
- Test if the 7th bit is set

Explain how to use bitwise operations to isolate the least significant bit (LSB) of a number in C.

Exercise 2:

Create a function countSetBits(int num) that counts and returns the number of set (1) bits in a given integer using bitwise operations

Exercise 3:

Implement a function reverseBits(int num) to reverse the binary representation of a 32-bit integer (i.e. 001 should become 100)